Suncorp // Databricks

Agentforce OpenTelemetry to Lakehouse Observability

Databricks Field Engineering

24 October 2026

Demonstration readout and test setup

This pack covers what was stood up in the Field Engineering lab, the live ingest that was demonstrated, and the **recommended Suncorp test setup** including **AWS Private Link** (Phase B), which this public FEVM workspace cannot prove.

Suncorp — recommended test setup

Suncorp wants Agentforce OTel traces in Databricks for centralised AI observability, on workspaces with **AWS Private Link enforced end-to-end**.

Split the proof into two environments. Do **not** try to prove Private Link on the Field Engineering FEVM lab — that workspace is public.

Phase	Where	What it proves	Status
A — Functional ingest	FEVM `fevm-ot-pdm-sam1` (this repo)	Agentforce → poll OTel JSON → Protobuf → Zerobus → Delta, no Kafka	**Done** (24 Oct 2026)
B — Private path	Suncorp PL workspace, same region as producers	Traffic never uses the public Zerobus hostname	**Needs Suncorp networking**

Phase B is extra network work. It does not require re-enabling Salesforce Session Tracing if Suncorp already has a Data Cloud + Agentforce org; it requires VPC endpoints, private DNS, and a collector in a Suncorp VPC.

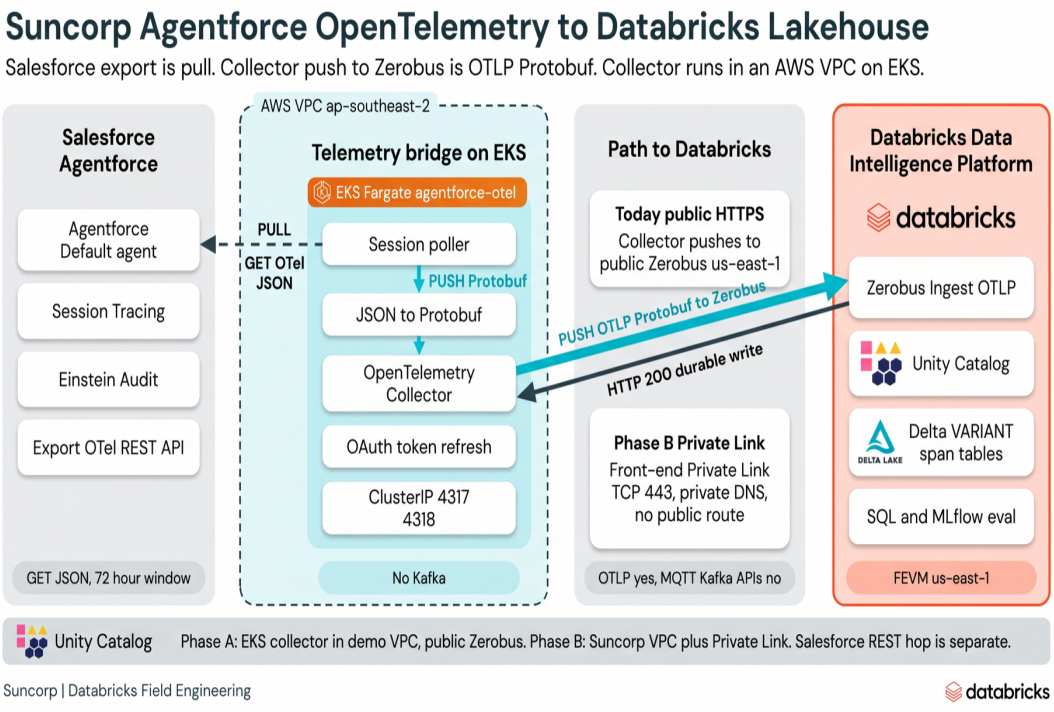


Figure 1: Suncorp Agentforce OpenTelemetry to Databricks Lakehouse

Phase A — already demonstrated (FEVM)

Live session `01a16cb0-5079-71ce-a37e-9aee0cb3c332` (utterance: *Hello, what can you do?*) produced **8 spans**. Zerobus returned HTTP **200**. Rows in `ot_pdm_semi_catalog.agentforce_otel.agentforce_otel_spans` included `GeneralCRM_16jaj000003iyMv`, `AiCopilot__ReactInitialPrompt`, `CLOSED_USER_REQUEST`, `TaskResolution`, `InstructionAdherence`.

Repeat on FEVM:

```
# Synthetic contract check
python3 scripts/send_direct_zerobus.py

# After a chat / sf agent test run (Data Cloud catch-up can take several minutes)
DRY_RUN=true python3 poller/poll_agentforce.py
OTEL_COLLECTOR_TRACES_URL="https://${WORKSPACE_ID}.zerobus.${REGION}.cloud.databricks.com:44
  python3 poller/poll_agentforce.py   # or scripts/send_direct_zerobus.py-style POST
SELECT time, service_name, name
FROM ot_pdm_semi_catalog.agentforce_otel.agentforce_otel_spans
ORDER BY time DESC
LIMIT 20;
```

Phase B — Private Link test (Suncorp must provision)

Zerobus OTLP **does** support front-end Private Link. MQTT and Kafka-compatible Zerobus APIs **do not**. See Zerobus networking and inbound PrivateLink for performance-intensive services (Zerobus listed on **TCP 443**).

What Suncorp networking needs to stand up

1. Workspace with **front-end Private Link** (existing Suncorp posture).
2. AWS **VPC endpoint** for workspace front-end and/or **inbound PL for performance-intensive services** in the **same region** as the workspace (Zerobus is regional).
3. **Private DNS** so `{region}.service-direct.privatelink.cloud.databricks.com` (and Zerobus names) resolve to the **VPC endpoint IP**, not public anycast.
4. Subnet security group: inbound **TCP 443** from the collector/poller security group.
5. Collector + poller in a **Suncorp VPC** with **no public route** to `*.zerobus.*.cloud.databricks.com`.
6. If Delta storage is firewalled: allowlist Databricks **serverless outbound IPs** for that region (Zerobus → table storage is a second hop).
7. Unity Catalog tables + table-level `SELECT`/`MODIFY` on a Suncorp catalog (same DDL as `sql/01_create_otel_tables.sql`, different three-part names).

Salesforce REST to `/einstein/audit/otel/{id}` is **not** covered by Databricks

Private Link. Privatise that hop separately with **Salesforce Private Connect** (or an approved egress to Salesforce API hostnames).

Phase B test cases (run from the VPC)

ID	Test	Pass criteria
PL-1	`dig` / `nslookup` of the service-direct / Zerobus PL hostname	Returns **private** VPC endpoint IP
PL-2	`nc -vz <pl-hostname> 443` from the collector subnet	Connects
PL-3	Same check against the **public** `*.zerobus.*.cloud.databricks.com` name from that subnet	**Fails** (no public route)
PL-4	OTLP Protobuf POST `/v1/traces` through the collector	HTTP 200
PL-5	SQL on the Suncorp UC span table	New rows for the test session
PL-6	(Optional) Salesforce poller to OTel API	200 with spans **or** documented Salesforce Private Connect path

Do not call Phase B complete unless **PL-3** fails and **PL-4/PL-5** succeed. That is the evidence Suncorp's "full Private Link" control is doing work.

Suggested owners

Work	Owner
VPC endpoint, DNS, SG, no public Zerobus route	Suncorp network / cloud
Workspace PL registration, serverless IP allowlist	Suncorp Databricks admin + Databricks SA
Collector + poller on EKS/EC2 in the VPC	Suncorp platform / FE overlay
Agentforce session + Data Cloud tracing	Suncorp Salesforce admin
Repeat ingest + SQL readout	Joint

Recommended sequence for Suncorp

1. Accept Phase A (this repo) as the **functional** proof — no message bus.
2. Book a Phase B window on a **non-prod PL workspace** in the producer region.
3. Deploy this repo's Collector YAML in the VPC (swap catalog/schema/workspace ID; keep Protobuf + `x-databricks-zerobus-table-name`).
4. Run PL-1 … PL-5 with a synthetic span first, then one live Agentforce chat.
5. Only then discuss HA (EKS), logs/metrics tables, and MLflow eval on VARIANT.

Out of scope until Phase B exists

- Proving "denied public Zerobus" from this laptop or FEVM
- Changing FEVM to Private Link
- End-to-end private Salesforce **and** Databricks in one hop without Salesforce Private Connect # Architecture — Agentforce OTel → Databricks Zerobus (Suncorp demo)

Bricksmith / Databricks-branded target architecture (Phase A data path plus Phase B Private Link):

Prompt used to generate the figure: suncorp_agentforce_otel_zerobus.txt (Bricksmith + Databricks brand style + Unity Catalog / Delta / Databricks logos).

Production target (four layers)

```
[ Salesforce / Agentforce ]
        |  Session Trace OTel REST API (OTLP JSON, beta)
        |  GET /services/data/v66.0/einstein/audit/otel/{sessionId}
        v
[ Telemetry bridge: poller + OpenTelemetry Collector ]
```

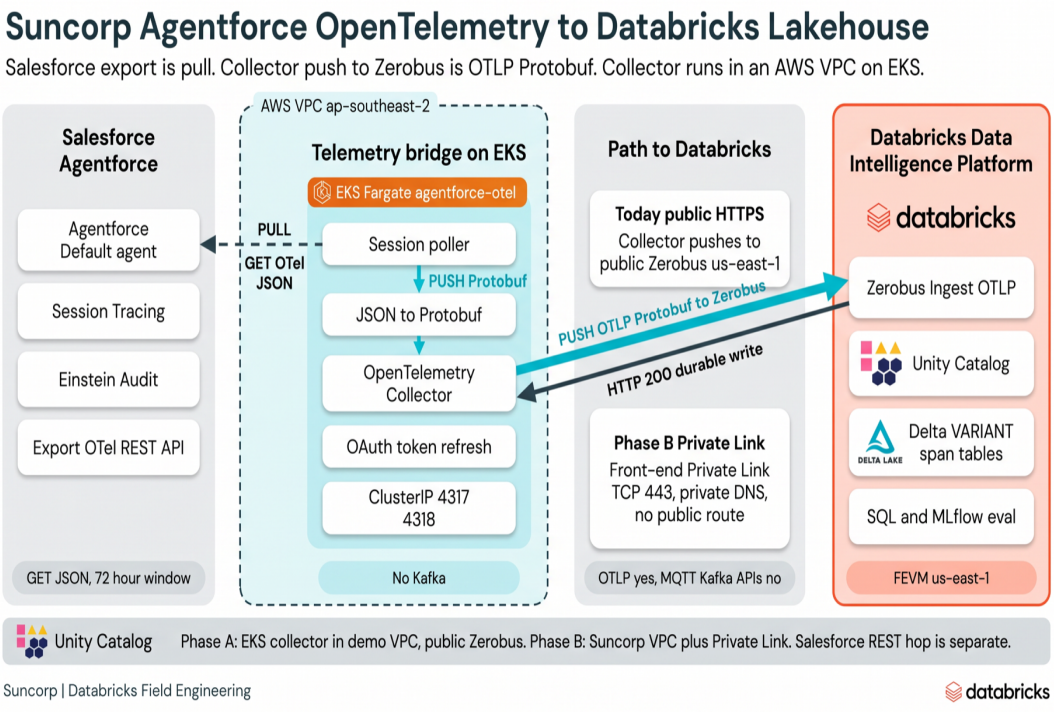


Figure 2: Suncorp Agentforce OpenTelemetry to Databricks Lakehouse

```
        |  Poller: discover sessions, JSON->Protobuf
        |  Collector in Suncorp VPC: oauth2clientauthextension token refresh
        |  OTLP Protobuf over Private Link (TCP 443)
        v
[ Databricks Zerobus Ingest OTLP Gateway ]   <- Unity Catalog authz
        |  Header: x-databricks-zerobus-table-name
        |  Direct managed write (no Kafka / Kinesis)
        v
[ Delta Lake tables (VARIANT attributes) ]
        |  Flat, query-ready spans / logs / metrics
        v
[ Lakehouse AI observability / eval / Zero-Copy back to SFDC ]
```

Suncorp workspaces enforce **front-end Private Link**. Zerobus OTLP can use that path; MQTT/Kafka-compatible Zerobus APIs cannot. See docs/suncorp-test-setup.md.

Why a Collector?

Databricks OAuth tokens for Zerobus expire about every **60 minutes**. The Collector's `oauth2clientauthextension` mints and refreshes tokens from a service principal without changing producer code. In production it also sits on the **private** hop (VPC → Private Link → Zerobus).

Why a Poller?

Agentforce does **not** push OTLP to an external endpoint today. The beta Session Trace OTel API returns one session as OTLP JSON. Production path is:

1. Agentforce writes session/LLM/tool data into Data 360 Session Tracing
2. Poller discovers `ssot__AiAgentSession__dlm` rows
3. Poller fetches each session's OTel payload
4. Poller converts JSON → Protobuf (Zerobus rejects `application/json`)
5. Poller posts to Collector → Zerobus → Delta

Salesforce evaluation spans may arrive with an **empty** `spanId`. The converter fills a deterministic 8-byte ID so Zerobus accepts the batch.

Lab vs production

	This repo (Phase A)	Suncorp production (Phase B)
Databricks	Public FEVM `fevm-ot-pdm-semi` us-east-1	PL-enforced workspace, same region as producers
Zerobus URL	`{workspaceId}.zerobus.us-east-1.cloud.databricks.com` (public)	Front-end / cloud Private Link, TCP 443

	This repo (Phase A)	Suncorp production (Phase B)
Collector	Optional; lab used **direct Protobuf POST** when Docker was down	Required, in Suncorp VPC, no public Zerobus route
Salesforce org	OrgFarm DE `orgfarm918`	Suncorp Agentforce + Data Cloud org
Proven	Live 8-span ingest, SQL on Delta	Not yet — needs network build

Demo environment (this repo)

Layer	Choice
Salesforce	OrgFarm Dev Edition `orgfarm-91850b6147-dev-ed`
Databricks	FEVM AWS Serverless `fevm-ot-pdm-semi` (us-east-1)
Catalog	`ot_pdm_semi_catalog.agentforce_otel`
Tables	`agentforce_otel_{spans,logs,metrics}`
Ingest used	Direct Zerobus OTLP/HTTP Protobuf (Collector YAML still in-repo for prod)
Auth	Databricks SP `agentforce-zerobus-otel` (table-level `SELECT+MODIFY`)
Live session	`01a16cb0-5079-71ce-a37e-9aee0cb3c332` (24 Oct 2026)

Key advantages for Suncorp

1. **No message bus** — no Kafka/MSK/Kinesis for lakehouse-bound telemetry
2. **Private Link compatible** — OTLP is a supported Zerobus PL interface
3. **Unified AI eval** — LLM-as-judge / RAG evals on live conversation traces
4. **Zero-Copy optionality** — share curated analytics back to Salesforce Data Cloud
5. **VARIANT columns** — preserve high-cardinality Agentforce prompt/tool metadata

References

- Zerobus OTLP overview
- Configure OTLP clients

- OTLP table reference
- Zerobus networking / front-end Private Link
- Inbound PrivateLink for performance-intensive services
- Agentforce Session Trace OTel API # Salesforce Agentforce setup (Org-Farm / Developer Edition)

Org used for this demo:

- Login URL: `https://orgfarm-91850b6147-dev-ed.develop.my.salesforce.com`
- Username: `pravin.varmadbrx.24b133d4e353@agentforce.com`
- Alias (sf CLI): `orgfarm918`
- Agent used for live traces: **Agentforce (Default)** / `Copilot_for_Salesforce`

Current org status (24 Oct 2026)

Check	Result
sf CLI login	Connected (`orgfarm918`)
Data Cloud	**Enabled**; Data Space `default` (`0vhaj000001VLKXAA4`) **ACTIVE**
Einstein Audit dataspace	**Bound to** `default` (Setup UI only — not Metadata API)
Agentforce Session Tracing	On
Agent Platform Tracing	On
Audit and Feedback	On (`enableAIFeedbackWithDC = true`)
Session Trace OTel API	**Healthy** — dummy UUID returns `404 session not found`
`ssot__AiAgentSession__dlm`	**Mapped and queryable**
`GenAIFeedback__dlm`	**Mapped and queryable**
SSOT entities	**153** after tracing semantic model finished
Live session ingested	`01a16cb0-5079-71ce-a37e-9aee0cb3c332` → 8 spans in Delta

CLI cannot set the Einstein Audit Data Space picker. After Data Cloud shows **Enabled**, open:

```
sf org open -o orgfarm918 -p lightning/setup/EinsteinAuditAnalyticsMonitoring/home
```

Hard-refresh, choose **default**, Save. Until that bind exists, OTel returns `No selected dataspace for Einstein Audit`.

Provisioning after the bind is sequential. OTel may 500 on missing `ssot__AiAgentSession__dlm` then `GenAIFeedback__dlm` for 15–60+ minutes.

1. Enable Agentforce

1. Setup → Quick Find → **Agentforce Settings**
2. Toggle **Enable Agentforce** → On (Default agent cannot be `sf agent activate`'d)

2. Enable Data 360 + Session Tracing + Audit

1. Setup → **Data Cloud Setup Home** — turn on Data 360 and wait until **Enabled**
2. Setup → **Einstein Audit, Analytics, and Monitoring Setup** (not the Agentforce model page)
3. Select Data Space `default` and Save
4. Confirm:
   - **Agentforce Session Tracing**
   - **Audit and Feedback**
   - **Agent Platform Tracing** if listed

Useful permsets on the running user: `GenieAdmin` (Data Cloud Architect), `CopilotSalesforceUser`, `CopilotSalesforceAdmin`. `CDPAdmin` may fail on DE (`CdpPsl` disabled); Genie starter PSL is the DE licence.

3. Activate Data 360 streams

In **Data Cloud → Data Streams**, ensure these are Active and refresh each:

- `AiAgentSession`
- `AiAgentInteraction`
- `AiAgentInteractionMessage`
- `AiAgentInteractionStep`

Until streams are mapped, OTel can 500 (`table does not exist`) or 404 even after chats. Data Cloud catch-up after a conversation was ~**6 minutes** in this lab.

4. Generate a traced session (CLI)

Default Agentforce agent is enough; a custom service agent is optional.

```
# One-off test spec → org (see /tmp or a local specs/ file)
sf agent test create --spec specs/OTel_Demo_Test.yaml \
  --api-name OTel_Demo_Test --target-org orgfarm918 --force-overwrite

sf agent test run --api-name OTel_Demo_Test --target-org orgfarm918 --wait 10 --verbose
```

Utterance used in the lab: `Hello, what can you do?`
Expected topic: `GeneralCRM`
Session id is in `generatedData.sessionId`.

Alternatively chat in Agent Builder / Copilot panel. Draft-only preview on an **inactive** custom bot often never lands in Session Tracing.

5. Poll OTel

`GET /services/data/v66.0/einstein/audit/otel/{session-id}`

- One session ID per request (no bulk yet)
- Sessions must have `StartTimestamp` within the prior **72 hours**
- Dummy UUID → `404 session not found` means the API is enabled
- Empty `spanId` on evaluation spans (`TaskResolution`, `InstructionAdherence`) is handled in `poller/json_to_otlp.py`

```
sf org display -o orgfarm918
DRY_RUN=true python3 poller/poll_agentforce.py
# Pin a session:
SALESFORCE_SESSION_ID=01a16cb0-5079-71ce-a37e-9aee0cb3c332 python3 poller/poll_agentforce.py
```

6. External Client App (recommended for unattended poller)

1. Setup → External Client Apps / App Manager
2. Enable **Client Credentials Flow**
3. API scope: **Manage user data via APIs** (`api`)
   Using only `full` can fail with `invalid_grant: no valid scopes defined`
4. Set **Run As** to a user with Data Cloud + Einstein Audit access
5. Copy Consumer Key / Secret into `SALESFORCE_CLIENT_ID` / `SALESFORCE_CLIENT_SECRET`

Until the ECA exists, the poller falls back to the `sf` CLI session for `orgfarm918`.